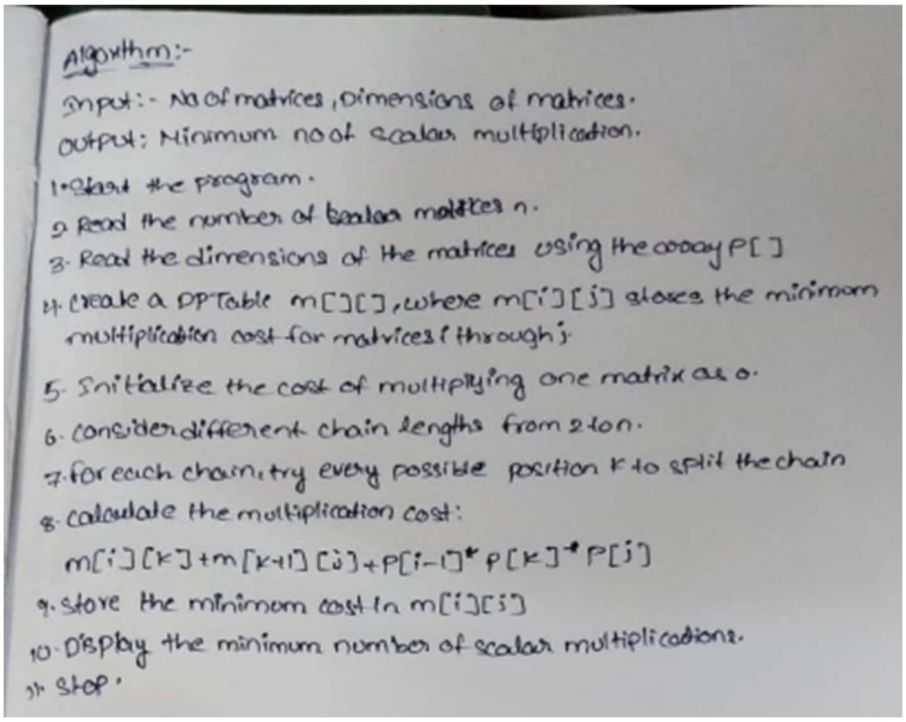
Practical-6

Aim: Implementation of Chain Matrix Multiplication using dynamic programming.

Algorithm:



Program:

```
#include <iostream>

using namespace std;


void matrixChainOrder(int p[], int n)

{
```

Name:B.Vennela
EnrollmentNo:92460118232

```
int m[20][20];

// Cost is zero when multiplying one matrix
for (int i = 1; i < n; i++)
   m[i][i] = 0;

// L is chain length
for (int L = 2; L < n; L++)
{
   for (int i = 1; i < n - L + 1; i++)
   {
      int j = i + L - 1;
      m[i][j] = 999999;

      for (int k = i; k < j; k++)
      {
         int cost = m[i][k] +
                  m[k + 1][j] +
                  p[i - 1] * p[k] * p[j];

         if (cost < m[i][j])
            m[i][j] = cost;
      }
   }
}
```

Name:B.Vennela
EnrollmentNo:92460118232

```
cout << "Minimum number of scalar multiplications = "
        << m[1][n - 1] << endl;
}
int main()
{
   // Dimensions of matrices:
   // A1 = 10x20
   // A2 = 20x30
   // A3 = 30x40
   // A4 = 40x30

    int p[] = {10, 20, 30, 40, 30};

  int n = sizeof(p) / sizeof(p[0]);

  matrixChainOrder(p, n);

  return 0;
}
```

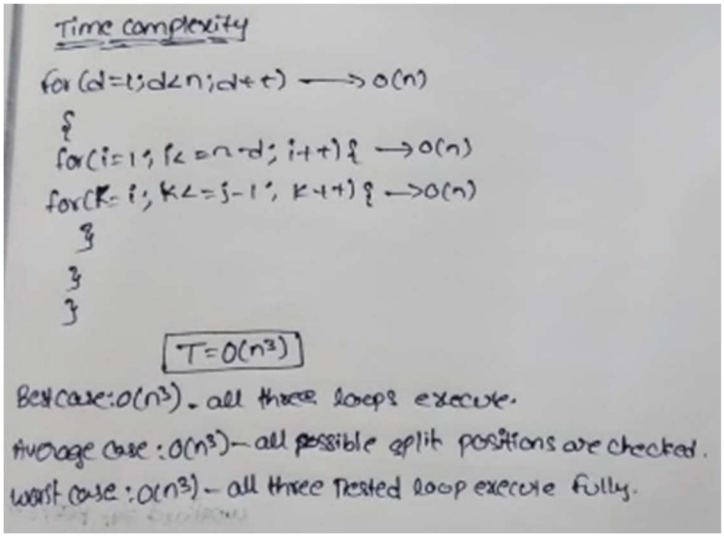
Output:

```
Minimum number of scalar multiplications = 30000

=== Code Execution Successful ===
```

Name:B.Vennela
EnrollmentNo:92460118232

Time complexity:



Conclusion: Chain Matrix Multiplication was successfully implemented using Dynamic Programming. It finds the best order to multiply matrices with the minimum number of calculations. This method saves time by avoiding repeated calculations and provides an efficient solution to the problem.

Name:B.Vennela
EnrollmentNo:92460118232